RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

www.rvce.edu.in
Tel: +91-80-68188100
+91-80-68188111
+91-80-68188112

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING ENGINEERING

Samveda : Neuroadaptive Learning Platform

Machine Learning Operations

REPORT

Submitted by

B Aishwarya Nayak	**1RV23AI127**
Mohit M	**1RV23AI056**
Yamini	**1RV23AI134**
Keerthi V C	**1RV23AI064**

Under the guidance of

Prof. Manasa M

Department of AIML

RV College of Engineering

In partial fulfilment for the award of degree of

Bachelor of Engineering

in

Department of Artificial Intelligence and Machine Learning

2024-2025

ABSTRACT

The rapid shift to online learning has revealed a major weakness in today's educational technology: it cannot truly understand whether students are engaged. Most platforms rely on basic indicators like attendance, quiz scores, or login time, which fail to capture real attention, focus, or comprehension. Non-verbal cues - such as eye movement, facial expressions, and head posture - are largely ignored. To address this gap, this project introduces **Samveda**, a neuroadaptive learning platform designed to sense student engagement in real time and respond intelligently when attention drops.

Samveda combines computer vision, machine learning, and modern MLOps practices to build an end-to-end intelligent system. Using MediaPipe for facial landmark detection and OpenCV for video processing, the platform extracts 2,458 features from webcam input, including eye aspect ratio, head pose, and texture-based facial patterns. These features are fed into an ensemble model made up of XGBoost, Random Forest, and Extra Trees classifiers. The model achieves a test accuracy of **63.32%** on the FER-2013 emotion recognition dataset, which is considered challenging due to noisy and imbalanced data.

A robust MLOps pipeline supports the entire lifecycle of the system. MLflow manages experiments and model versions, Prometheus and Grafana enable real-time monitoring, Evidently AI tracks data quality and drift, PostgreSQL stores structured data, and Docker ensures reliable deployment. When low engagement is detected, Samveda pauses the learning content and presents context-aware questions to re-engage the learner. This project demonstrates how production-ready MLOps systems can turn AI models into meaningful, adaptive tools for online education.

Chapter 1

Introduction to Samveda: Neuroadaptive Learning Platform

1.1 Background and Motivation

The COVID-19 pandemic accelerated the shift to online learning, ensuring continuity but exposing a key limitation of e-learning platforms - the lack of real-time insight into student engagement. Unlike physical classrooms, online environments rely on indirect metrics such as login time and quiz scores, which reflect outcomes rather than actual attention or cognitive effort, delaying timely intervention. Advances in computer vision and machine learning now make engagement detection feasible, yet most solutions remain research prototypes without production robustness. This project bridges that gap by integrating real-time engagement detection with production-grade MLOps practices. **Samveda** actively responds to disengagement by adapting learning content in real time, restoring meaningful feedback and responsiveness to online learning. Grounded in cognitive science and ethical AI principles, Samveda represents a deployable, engagement-aware learning platform..

1.2 Problem Statement

Online learning platforms lack real-time mechanisms to measure and respond to student engagement, allowing disengagement to go unnoticed and resulting in poor comprehension, low retention, and higher dropout rates. Existing systems rely on passive indicators such as video completion, which often misrepresent actual attention and ignore rich non-verbal cues like facial expressions and gaze. Educators receive only delayed, aggregated feedback, limiting timely intervention. Although engagement detection has been widely explored in research, most solutions fail to scale due to the absence of production-grade MLOps practices. This project addresses these limitations by developing a real-time, webcam-based engagement classification system supported by a robust MLOps pipeline, enabling adaptive interventions, low-latency inference, and reliable continuous deployment.

1.3 Objectives of the Project

The primary goal of this project is to develop a production-ready neuroadaptive learning platform that detects and responds to student engagement in real time.

Key objectives include:

- Designing a real-time engagement detection system using MediaPipe-based facial landmarks and a 2,458-dimensional feature set
- Maintaining inference latency below 100 ms for real-time usability
- Implementing a full MLOps pipeline using MLflow, Prometheus, Grafana, Evidently AI, Docker, and PostgreSQL
- Building an adaptive intervention mechanism that pauses content and prompts students with engagement-restoring questions
- Ensuring transparency through SHAP and LIME-based explainability
- Delivering a user-friendly Streamlit interface with real-time feedback and analytics

Secondary objectives include performance validation, risk mitigation, documentation, production-readiness testing, and future extensibility.

1.4 Significance and Novelty

This project contributes to both educational technology and applied MLOps by making student engagement measurable and actionable. **Samveda** enables proactive, personalized interventions through real-time engagement awareness, supporting learner self-reflection and metacognitive development.

Technically, the system serves as a full-stack MLOps reference, integrating experiment tracking, monitoring, drift detection, explainability, and real-time deployment. It demonstrates that interpretable classical machine learning models, combined with effective feature engineering, can deliver reliable real-time performance on consumer hardware.

Key novelties include a closed-loop adaptive intervention mechanism, explainability-first design, edge-focused MLOps deployment, behavioral drift monitoring, and a modular Docker-based architecture.

Chapter 2

Literature Survey and Related Work

2.1 Engagement Detection in E-Learning

Student engagement is widely recognized as a key determinant of learning effectiveness, encompassing behavioral, emotional, and cognitive dimensions (Fredricks et al., 2004; Pekrun & Linnenbrink-Garcia, 2012). With the expansion of online education, researchers have increasingly focused on automated methods to identify engagement in digital learning environments.

Early e-learning research relied on interaction-based indicators such as login frequency, click patterns, time spent on learning materials, and assessment performance. While these measures provide useful behavioral insights, they offer only indirect evidence of actual engagement and fail to capture learners' emotional or attentional states (D'Mello & Graesser, 2012).

Physiological sensing approaches, including EEG and heart rate monitoring, have been explored to obtain more direct engagement signals (Picard & Daily, 2005). However, these methods are costly, intrusive, and unsuitable for large-scale educational deployment.

Computer vision–based techniques offer a practical alternative by using webcams to analyze facial and behavioral cues related to engagement. Prior studies have shown that eye gaze, blink rate, head movements, and facial expressions correlate with levels of attention and affect. Despite promising results, most systems remain research prototypes and lack the infrastructure required for real-time deployment and long-term reliability.

2.2 Computer Vision–Based Emotion Recognition

Facial emotion recognition forms the foundation of vision-based engagement analysis. Seminal work by Ekman and Friesen (1971) demonstrated that certain facial expressions are universally associated with basic emotions, providing a theoretical basis for automated recognition systems.

Traditional emotion recognition pipelines employ face detection, landmark localization, feature extraction, and classification. Commonly used features include geometric relationships between facial landmarks, texture descriptors such as Local Binary Patterns, and frequency-based representations using Gabor filters. Benchmark datasets such as FER-2013 (Goodfellow et al., 2013) and AffectNet (Mollahosseini et al., 2017) have accelerated progress while highlighting challenges arising from low-resolution images, pose variations, and label noise.

Although deep learning approaches, particularly convolutional neural networks, have improved classification accuracy, they require large labeled datasets and significant computational resources. Additionally, their limited interpretability poses challenges in educational settings. As a result, hybrid approaches combining handcrafted features with ensemble classifiers—such as Random Forests (Breiman, 2001) and XGBoost (Chen & Guestrin, 2016)—remain relevant due to their interpretability and robustness with limited data.

2.3 MLOps Practices for Educational ML Systems

Machine Learning Operations (MLOps) addresses the gap between experimental models and real-world deployment by emphasizing reproducibility, monitoring, and maintainability (Sculley et al., 2015). In contrast to research-focused evaluations, production systems must handle evolving data, infrastructure failures, and performance degradation.

Key MLOps components include experiment tracking, automated deployment pipelines, model monitoring, and data quality validation. Tools such as MLflow support systematic experiment management and model versioning (Zaharia et al., 2018), while monitoring frameworks enable real-time visibility into system health. Drift detection techniques are essential for identifying changes in input data distributions that can degrade model performance (Breck et al., 2017).

Explainability is another critical requirement for trustworthy educational systems. Model-agnostic explanation techniques such as LIME (Ribeiro et al., 2016) and SHAP (Lundberg & Lee, 2017) help interpret predictions, supporting transparency and user trust.

2.4 Existing Systems and Limitations

Several existing solutions address engagement-related challenges from different perspectives. Proctoring systems primarily focus on academic integrity rather than learning support, often raising privacy concerns. Learning analytics platforms rely on interaction data and operate at coarse temporal resolutions, limiting their responsiveness. Adaptive learning systems adjust content difficulty based on assessments but cannot distinguish disengagement from conceptual misunderstanding.

Academic prototypes using facial analysis demonstrate feasibility but rarely incorporate production-ready engineering practices, privacy-preserving designs, or long-term monitoring. Across these systems, common limitations include indirect engagement measurement, lack of real-time intervention, limited explainability, and insufficient deployment infrastructure.

2.5 Research Gaps

Based on the reviewed literature, the following gaps remain unaddressed:

1. Lack of production-grade, real-time engagement detection systems
2. Absence of closed-loop adaptive interventions
3. Limited student-facing explainability
4. Privacy concerns due to centralized data processing
5. Underutilization of interpretable feature-engineering approaches
6. Limited guidance on integrating multiple MLOps tools into a unified system

Chapter 3

Solution Design: Architecture, Algorithms, and Implementation

3.1 Overall System Architecture

Samveda adopts a modular, microservice-oriented architecture designed to support real-time engagement detection while ensuring privacy, scalability, and maintainability. The system is organized into three conceptual layers: the user-facing application layer, the machine learning inference layer, and the MLOps infrastructure layer.

At runtime, the system operates as a continuous feedback loop. Webcam video is processed locally to detect facial landmarks, from which a high-dimensional feature vector is extracted. These features are passed to an ensemble machine learning model that predicts the learner's engagement level. When disengagement is detected, intervention logic pauses the learning content and prompts reflective questions. All predictions and system events are logged for monitoring, drift detection, and future analysis.

This pipeline executes continuously at real-time frame rates, with careful optimization to ensure end-to-end latency remains within interactive limits.

The architecture follows several guiding principles. Modularity and separation of concerns allow individual components to be developed, tested, and updated independently. Event-driven communication minimizes tight coupling between services, while observability is treated as a first-class requirement through integrated logging and metrics. Privacy is prioritized by ensuring that raw video frames never leave the local device, and the system is designed to fail safely under errors such as camera loss or service unavailability.

3.2 Component Design

3.2.1 Frontend Application

The frontend application provides the primary user interface, handling webcam capture, learning content display, real-time engagement feedback, and intervention prompts. It also allows users to manage sessions and access explainability outputs when required.

The interface is implemented using a lightweight Python-based framework that supports reactive updates and persistent session state. Video processing is handled asynchronously to prevent UI blocking, ensuring smooth interaction throughout the learning session.

3.2.2 Computer Vision Pipeline

The computer vision pipeline is responsible for face detection and facial landmark extraction from video frames. A dense facial mesh representation enables fine-grained analysis of facial movements relevant to engagement and emotion estimation.

The pipeline is optimized for CPU-based execution on consumer hardware and includes safeguards that gracefully handle temporary detection failures by skipping problematic frames rather than interrupting execution.

3.2.3 Feature Engineering

A comprehensive feature extraction module converts raw facial landmarks and images into a 2,458-dimensional feature vector. Features include geometric measurements, texture descriptors, gradient-based features, and statistical summaries computed across facial regions.

All features are normalized to ensure numerical stability and consistent model input. The feature engineering process emphasizes interpretability, allowing individual dimensions to be mapped to meaningful facial or statistical properties. Robust error handling ensures that invalid or incomplete inputs do not propagate failures downstream.

3.2.4 Machine Learning Models

Samveda employs an ensemble classification approach combining gradient-boosted trees, random forests, and extremely randomized trees. Each model contributes complementary strengths in handling structured features, robustness to noise, and generalization performance.

Predictions are generated using soft voting, enabling probability-based confidence estimation. The models are trained on benchmark emotion datasets and mapped to engagement states using a custom rule-based transformation. Trained models are serialized and loaded once during service startup to minimize inference overhead.

3.2.5 Model Serving and Deployment

Model inference is exposed through a RESTful API implemented using a lightweight backend framework. The service handles feature scaling, prediction generation, confidence scoring, and health monitoring.

Containerization ensures reproducible deployment and dependency isolation. The deployment design favors simplicity and reliability, avoiding unnecessary orchestration complexity while remaining extensible for future scaling.

3.2.6 Data Storage

A relational database is used to store session metadata, engagement predictions, intervention events, and selected feature vectors for monitoring and drift analysis. Flexible storage of high-dimensional data is achieved using structured JSON fields, while indexing strategies support efficient time-based queries.

The schema design balances strict relational integrity with flexibility, supporting both operational queries and offline analytical workflows.

3.2.7 Experiment Tracking

All training experiments are systematically logged using an experiment tracking framework. Logged artifacts include model parameters, performance metrics, trained models, and visualization outputs such as confusion matrices and feature importance plots.

This setup ensures full reproducibility and enables structured comparison across experimental configurations.

3.2.8 Monitoring and Observability

System observability is achieved through comprehensive metric collection and visualization. Metrics capture inference latency, face detection performance, prediction distributions, database activity, and intervention frequency.

Dashboards provide real-time insights into system health and long-term trends, supporting proactive maintenance and performance tuning.

3.2.9 Data Quality and Drift Detection

To maintain long-term reliability, the system performs periodic data quality checks and drift detection analyses. Statistical tests are applied to compare incoming feature distributions against reference training data, as well as to detect shifts in prediction outputs.

Automated alerts are triggered when drift exceeds configurable thresholds, signaling the need for retraining or system review.

3.2.10 Model Explainability

Model explainability is supported at both global and local levels. Global analyses identify the most influential features across predictions, while local explanations clarify why specific predictions were made.

Explanations are generated on demand to minimize computational overhead and are presented in a form accessible to non-expert users, promoting transparency and trust.

3.3 Technology Stack Selection

The technology stack for Samveda was selected based on production readiness, ease of integration, community maturity, licensing, and suitability for edge-based deployment. Each component was chosen to balance performance, interpretability, scalability, and privacy.

Python 3.10 was selected due to its dominance in the machine learning ecosystem and extensive library support. Version 3.10 provides performance and stability improvements while remaining fully compatible with all required ML and MLOps frameworks.

MediaPipe (Google) was chosen for facial detection and landmark extraction because of its high efficiency on CPU-based consumer hardware. Its optimized pipelines and pre-trained models enable real-time performance without additional training. Alternatives such as Dlib, Haar Cascades, and MTCNN were evaluated but rejected due to lower accuracy, higher latency, or heavier dependencies.

OpenCV (cv2) serves as the core image and video processing library. Its maturity, optimization, and extensive documentation make it well suited for real-time computer vision tasks within the system.

Ensemble machine learning models (XGBoost, Random Forest, Extra Trees) were selected for their strong performance on structured features, fast inference, robustness to noise, and high interpretability. XGBoost contributes regularization and speed, Random Forest offers stability, and Extra Trees increase model diversity. Deep learning models were excluded due to higher computational cost, lower explainability, and limited suitability for low-resource edge devices.

MLflow was adopted for experiment tracking and model management because of its open-source nature, strong ecosystem support, and seamless integration with Python-based ML workflows. Cloud-dependent tools such as Weights & Biases and Neptune.ai were avoided to reduce privacy risks and external dependencies.

Prometheus and Grafana were selected for monitoring and observability. Prometheus provides reliable time-series metric collection with strong container support, while Grafana enables intuitive visualization of system health, performance, and model behavior. Commercial monitoring tools were excluded due to cost and vendor lock-in.

Evidently AI was incorporated for data quality validation and drift detection. Its built-in statistical tests and clear visual reports make it effective for monitoring feature and prediction distribution changes in production systems.

PostgreSQL was chosen as the primary database due to its ACID compliance, reliability, and support for JSONB fields, allowing flexible storage of engagement features while maintaining powerful relational querying.

Docker enables reproducible deployments, dependency isolation, and modular service design aligned with the system's microservice-style architecture. Full orchestration platforms such as Kubernetes were considered unnecessary for the current deployment scale.

FastAPI was selected for backend services because of its high performance, automatic API documentation, strong type validation, and suitability for production-grade inference services.

Streamlit was used to build the frontend interface due to its rapid development capabilities, minimal frontend overhead, and support for interactive educational dashboards.

SHAP and LIME were integrated to support model explainability. SHAP provides theoretically grounded global and local explanations for tree-based ensembles, while LIME offers model-agnostic, instance-level interpretability. Together, they enhance transparency and user trust.

Overall, the selected technology stack achieves a balanced trade-off between performance, interpretability, development efficiency, and deployability. All components are open-source, ensuring cost effectiveness, transparency, and freedom from vendor lock-in while meeting the constraints of edge-based educational systems.

3.4 Design Considerations

The design of Samveda was guided by ethical, technical, and usability requirements. All webcam processing is performed locally, and raw video data is neither stored nor transmitted, ensuring strong privacy protection. Only extracted features and engagement predictions are logged. Model behavior is made transparent using SHAP and LIME, and users retain control over tracking and interventions. Dataset biases are acknowledged, with mitigation planned as future work.

The system is optimized for real-time performance, delivering predictions within 100 ms using CPU-efficient tools suitable for low-end devices. A containerized, stateless architecture supports future horizontal scalability. Maintainability is ensured through a modular design, externalized configuration, structured logging, and full version control using Git and MLflow.

Samveda is designed to be robust, degrading gracefully during failures such as face detection or database issues. Health checks and fail-safe defaults prevent crashes. The user interface prioritizes

minimal friction, clear feedback, and supportive interventions, ensuring practical usability in real educational settings.

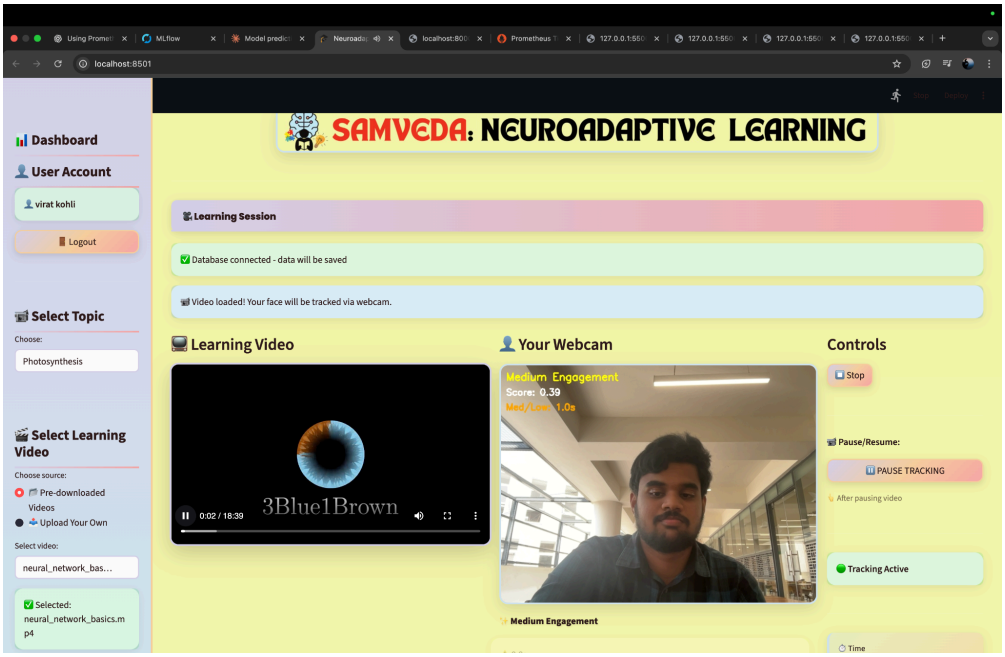


Figure 3.1 - User Interaction

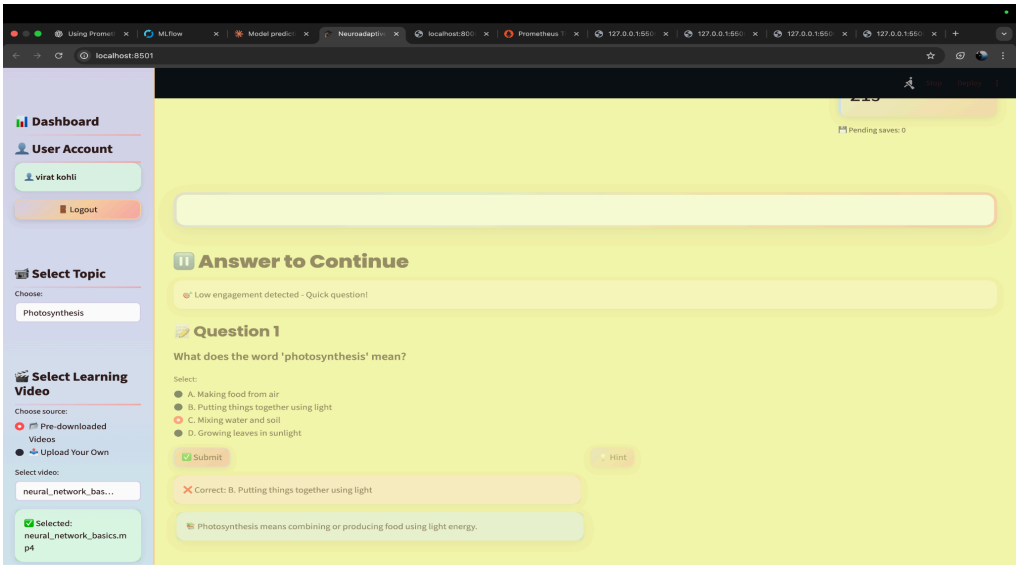


Figure 3.2 - System Response

Chapter 4

Dataset and Feature Engineering

4.1 Dataset Description

Samveda is trained using the FER-2013 dataset, a widely used benchmark for facial emotion recognition. It contains approximately 35,887 grayscale images of size 48×48 pixels, labeled into seven emotion classes: anger, disgust, fear, happiness, sadness, surprise, and neutral. The dataset follows a predefined split of 80% training, 10% validation, and 10% testing.

Images were collected from web sources, resulting in diverse "in-the-wild" data. Consequently, the dataset exhibits challenges such as low resolution, label ambiguity, class imbalance, occlusions, lighting variations, pose diversity, and demographic bias. Despite these limitations, FER-2013 is considered a realistic benchmark, and models that perform well on it tend to generalize better to real-world conditions.

Emotion-to-Engagement Mapping

To align emotion recognition with engagement estimation, the seven emotion classes were mapped into three engagement levels:

- **High Engagement:** Happy, Surprise
- **Medium Engagement:** Neutral, Angry, Fear
- **Low Engagement:** Sad, Disgust

This simplified mapping is supported by educational psychology literature and provides a practical abstraction for real-time engagement detection.

4.2 Data Collection and Preprocessing

All images are loaded in grayscale format, resized if necessary, and normalized to the **[0,1]** intensity range. Defensive loading mechanisms handle unreadable or corrupt samples without interrupting the pipeline. Emotion labels are numerically encoded and mapped to engagement classes.

The original FER-2013 train–test split is preserved for reproducibility, with a stratified validation split applied to maintain class balance. After preprocessing, the dataset contains approximately 25,800 training, 2,800 validation, and 3,600 test samples. Moderate class imbalance toward medium engagement is addressed during training using class weighting.

Data augmentation techniques were evaluated but excluded, as FER-2013 already provides substantial variability and ensemble models showed minimal benefit from augmentation relative to computational cost.

4.3 Feature Extraction Using OpenCV and MediaPipe

Samveda employs an extensive feature engineering pipeline that converts low-resolution facial images into a 2,458-dimensional feature vector. Facial landmarks are extracted using MediaPipe Face Mesh, which detects 468 three-dimensional landmarks representing key facial regions.

From these landmarks, geometric features such as eye aspect ratio and head orientation (pitch, yaw, roll) are computed to capture attention-related cues. Texture and appearance features, including Histogram of Oriented Gradients (HOG), Local Binary Patterns (LBP), and Gabor filter responses, capture fine-grained expression patterns.

Additionally, statistical and regional features summarize global pixel distributions and region-specific characteristics from the eyes, nose, and mouth. Symmetry and edge-based descriptors improve robustness to noise and occlusions. All features are concatenated into a single, interpretable representation.

4.4 Feature Scaling and Selection

Prior to model training, all features are standardized to zero mean and unit variance. Although feature selection techniques were explored, all 2,458 features were retained, as tree-based ensemble models perform implicit feature selection effectively. Retaining the full feature set preserves complementary signals and supports detailed explainability using SHAP and LIME.

Post-training analysis confirms that while HOG features contribute most strongly, statistical, geometric, and texture features collectively influence prediction.

Chapter 5

Machine Learning Model Development

5.1 Algorithm Selection and Justification

The choice of machine learning algorithms directly affects prediction accuracy, inference latency, interpretability, and suitability for edge deployment. Multiple approaches were evaluated before selecting the final ensemble-based architecture.

Convolutional Neural Networks (CNNs) were considered due to their strong performance on image data. However, they require large datasets, GPU resources, and exhibit poor interpretability and higher CPU inference latency. These constraints make CNNs unsuitable for privacy-preserving edge deployment and transparent educational systems; hence, they were rejected.

Support Vector Machines (SVMs) were also evaluated because of their effectiveness in high-dimensional spaces. However, their inference cost with non-linear kernels and limited scalability to thousands of features made them impractical for real-time use.

Tree-based models were identified as the most suitable compromise. Random Forests offer fast inference, robustness to noise, and built-in feature importance but may underfit complex patterns. XGBoost provides strong accuracy through gradient boosting and regularization, while Extra Trees increase diversity through aggressive randomization.

Final Selection:
An ensemble combining **XGBoost, Random Forest, and Extra Trees** was selected. Predictions are combined using **soft voting**, allowing probability averaging and reducing individual model errors through ensemble diversity.

5.2 Model Training and Hyperparameter Tuning

Each base learner was trained independently using scaled feature inputs. Hyperparameters were optimized using randomized search with cross-validation to balance performance and computational efficiency. Early stopping was applied where applicable to limit overfitting.

Randomized hyperparameter tuning was preferred over exhaustive grid search to efficiently explore the search space. All training runs, configurations, and artifacts were logged using MLflow to ensure reproducibility and traceability. The final ensemble model assigns higher weight to XGBoost due to its superior standalone performance.

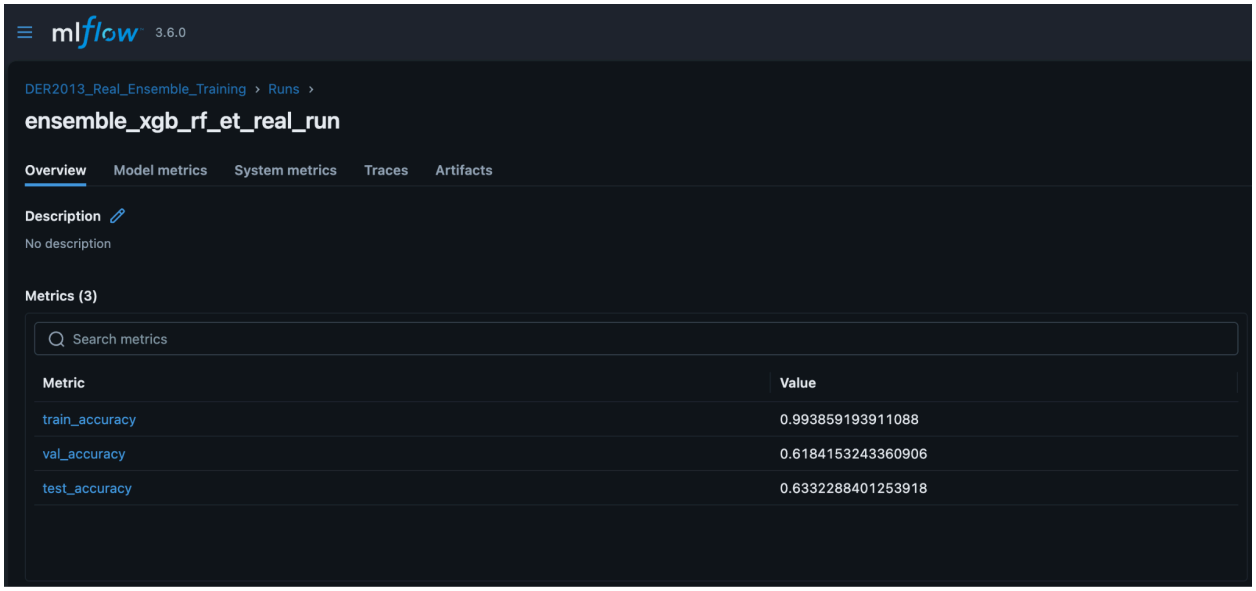


Figure 5.1 - MLFlow Interface

Observed Performance:

- Training Accuracy: **99.38%**
- Validation Accuracy: **61.84%**
- Test Accuracy: **63.32%**

The train–validation gap indicates mild overfitting; however, consistent test performance confirms acceptable generalization.

5.3 Model Evaluation Metrics

Model evaluation was conducted using multiple metrics suitable for multi-class engagement classification.Accuracy on the test set was 63.32%, providing a strong baseline given the difficulty of FER-2013.

A confusion matrix was analyzed to understand class-wise behavior.

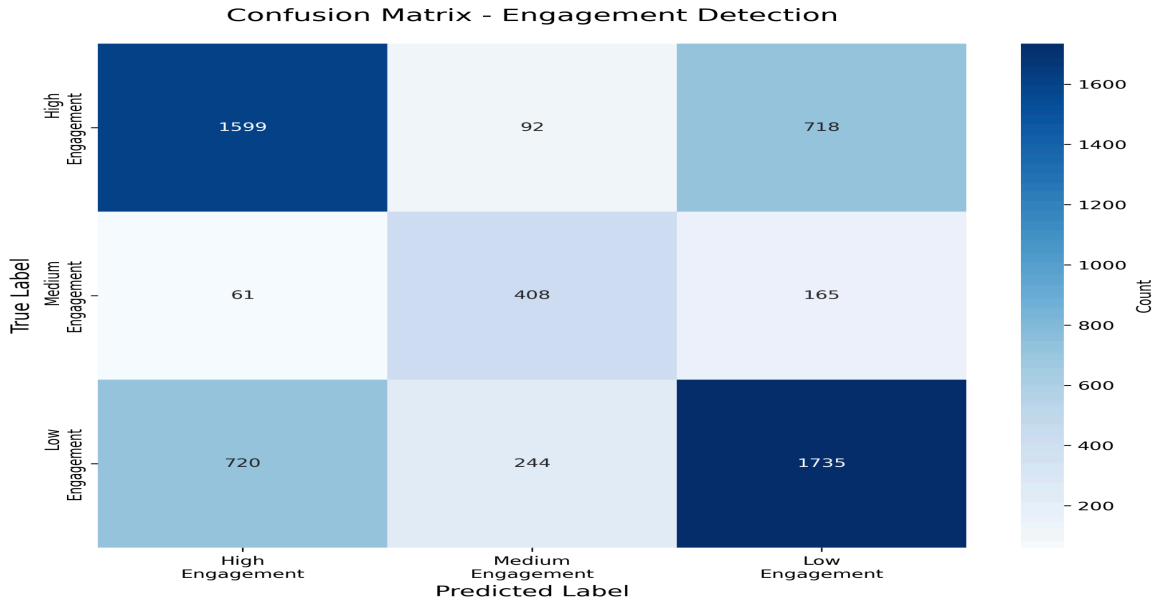


Figure 5.2 - Confusion Matrix of Engagement Detection

Key observations include strong separation of High and Low engagement classes, while Medium engagement remains harder to classify due to its ambiguous nature.Precision, Recall, and F1-score further highlight balanced performance across classes, with F1-scores ranging from 0.59 to 0.66.

To evaluate discriminative ability, ROC-AUC (One-vs-Rest) was computed, yielding a value of 0.78, indicating good class separability. Log loss of 0.92 confirms reasonably well-calibrated probability estimates.

5.4 Comparative Analysis of Models

A comparative evaluation was conducted between individual models and the ensemble:

- Random Forest: 61.8% test accuracy
- XGBoost: 62.9% test accuracy
- Extra Trees: 61.2% test accuracy
- Ensemble: 63.3% test accuracy

Although the ensemble has slightly higher inference latency (18 ms) than individual models (8–12 ms), it remains well below the real-time constraint of 100 ms. The ensemble consistently generalizes better, justifying its selection despite added complexity. Considering that human agreement on FER-2013 is approximately 65–70%, and state-of-the-art deep learning models typically achieve 65–75%, the achieved performance is competitive for a non-deep, interpretable, edge-deployed system.

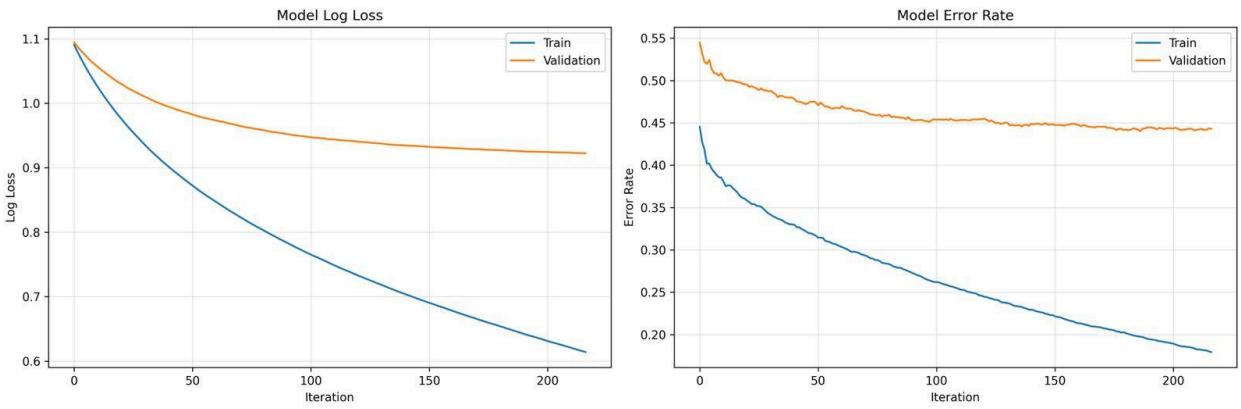


Figure 5.3 - SMOTE Analysis

5.5 Final Model Selection

Based on performance, robustness, interpretability, and deployment feasibility, the **XGBoost + Random Forest + Extra Trees ensemble** was selected as the production model. It provides the highest test accuracy, reduced variance through ensemble averaging, compatibility with SHAP and LIME for explainability, and efficient CPU-based inference without GPU dependency.

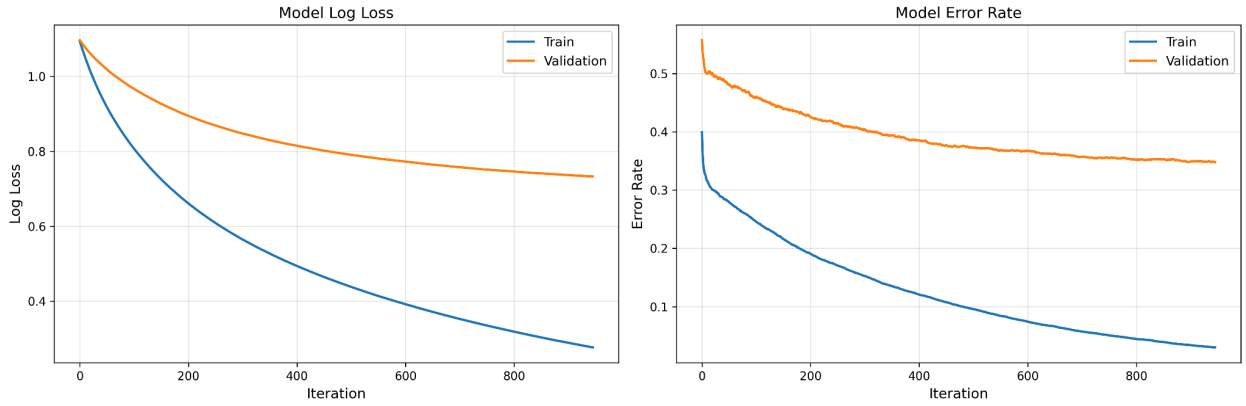


Figure 5.4 - Model Performance Graph

The training curves show stable convergence with decreasing log loss and error rates, along with mild but acceptable overfitting. Overall, the model satisfies both technical and operational requirements for real-time engagement detection.

Chapter 6

MLOps Implementation

6.1 Experiment Tracking with MLflow

MLflow is used as the central platform for managing the machine learning lifecycle in Samveda, supporting experiment tracking, model versioning, and artifact management.

A standalone MLflow tracking server is deployed with a local backend store and artifact repository, making it suitable for edge-based development environments. All experiments are organized under a dedicated namespace for engagement detection models.

Each training run logs key metadata, including model hyperparameters, dataset information, feature dimensionality, and class balancing strategies. Performance metrics such as accuracy and F1-score are recorded across training, validation, and test sets. Visual artifacts, including confusion matrices and feature importance plots, are also stored to aid qualitative analysis. Contextual tags improve experiment organization and traceability.

Trained models are registered in the MLflow Model Registry, where version history and lifecycle stages (Staging and Production) are maintained. Only models that satisfy predefined performance criteria are promoted for deployment.

Key benefits of this setup include full reproducibility, easy comparison across experiments, collaborative visibility, and a complete audit trail of modeling decisions.

6.2 Model Versioning and Registry

MLflow's model registry enables structured version control of trained models. Each improved model is registered as a new version, allowing systematic comparison and safe promotion. In Samveda, earlier Random Forest and XGBoost models were retained for reference, while the ensemble model was promoted to Production based on superior test accuracy.

Model lifecycle stages - None, Staging, Production, and Archived—ensure controlled deployment and straightforward rollback in case of regressions.

6.3 Pipeline Automation

Samveda follows a clear and repeatable pipeline structure for both training and inference, implemented through modular scripts rather than automated orchestration tools.

The training pipeline consists of sequential stages for data loading, feature engineering, model training, evaluation, and registration. Each stage is independently executable, improving reproducibility and debugging. All outputs are logged and versioned using MLflow.

The real-time inference pipeline processes webcam frames through face detection, feature extraction, and engagement prediction in a single pass, producing engagement labels with confidence scores and timestamps. This design ensures low latency and supports independent component upgrades.

Although orchestration tools such as Airflow are not currently used, the pipeline structure enables future automation without architectural changes.

6.4 Monitoring and Drift Detection

To ensure reliable real-time performance, Samveda integrates continuous monitoring and drift detection.

Prometheus collects custom metrics exposed through a `/metrics` endpoint, including inference latency, prediction counts by engagement level, and active session statistics. Instrumentation is applied directly around inference logic to enable precise performance monitoring.

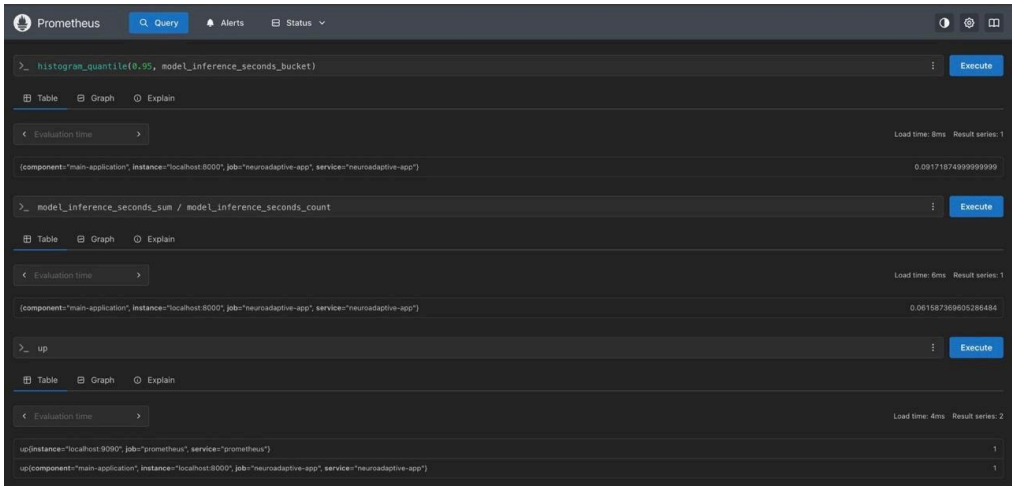


Figure 6.1 - Prometheus Dashboard

Latency analysis confirms that prediction times remain within real-time limits, with 95th percentile latency below 100 ms.

Grafana visualizes these metrics through real-time dashboards that track application health, CPU usage, inference latency, uptime, and memory stability.

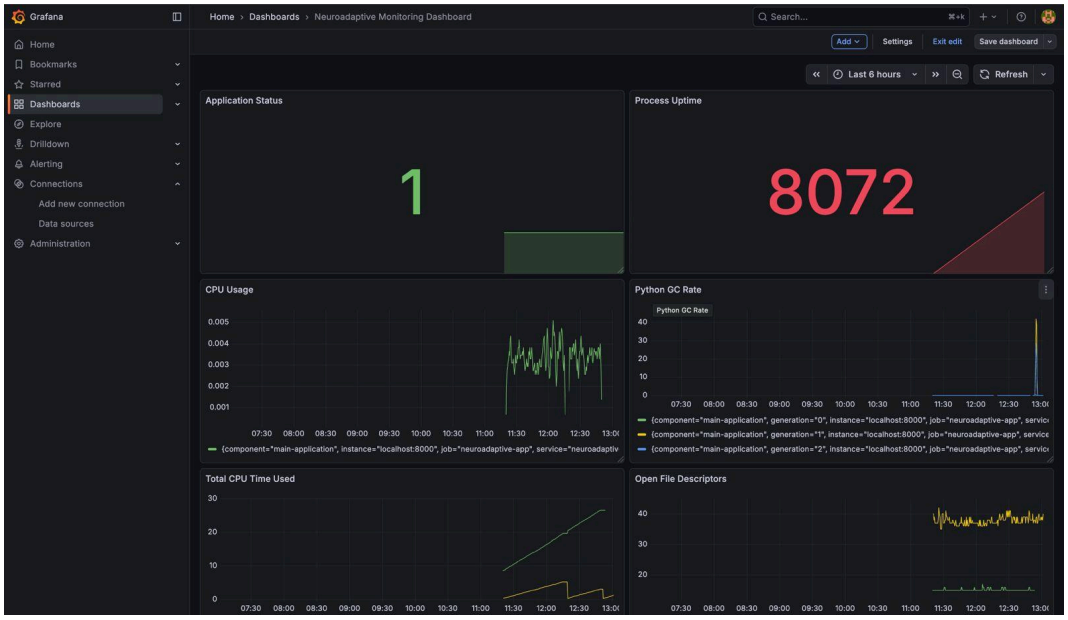


Figure 6.2 - Grafana Dashboard

For model reliability, Evidently AI is used to perform data quality checks and drift detection. These checks identify missing values, duplicates, and feature distribution shifts. Observed drift remains low and does not require immediate retraining.

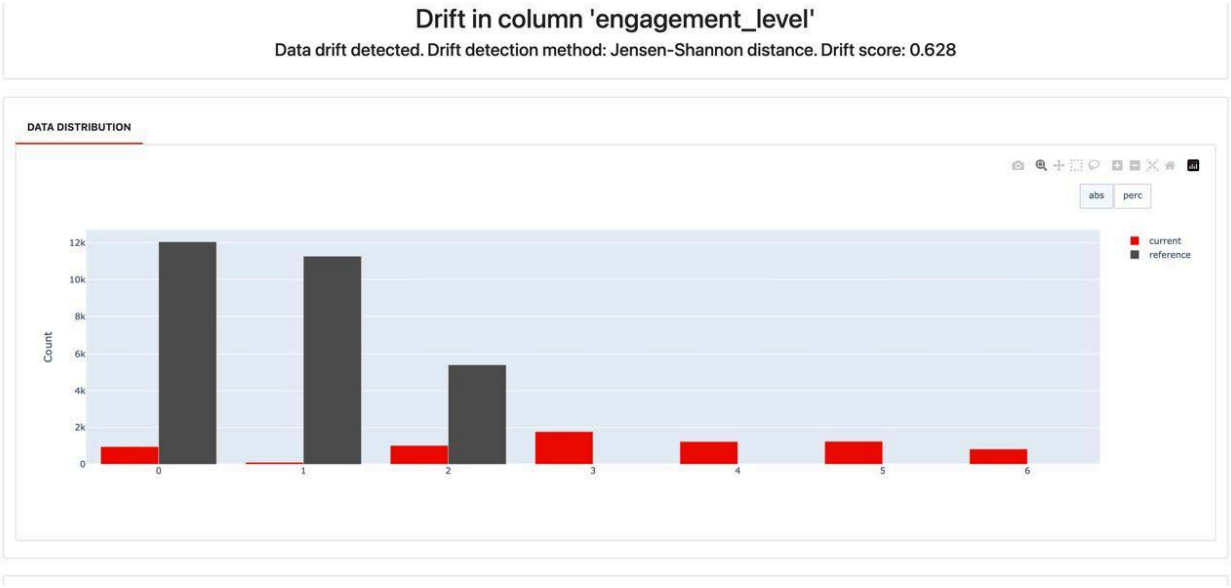


Figure 6.3 - Evidently AI Dashboard

Alert thresholds are defined for key conditions such as high inference latency, increased error rates, significant data drift, and service downtime. Prometheus-based alerts enable early detection and corrective action before user experience is affected.

6.5 Model Deployment Using Docker

Docker is used to enable reproducible and isolated deployment of the Samveda model inference service. The model API is packaged into a lightweight container containing only essential dependencies, minimizing image size and startup time. Health checks are included to continuously verify service availability.

Service orchestration is managed using Docker Compose, allowing controlled startup, automatic restarts, and secure mounting of trained model artifacts. Runtime configuration is handled using environment variables, ensuring flexibility across environments.

The deployment workflow includes image building, service startup, health monitoring, log inspection, and graceful shutdown. This approach ensures consistent deployment across development and production setups while maintaining observability and ease of maintenance.

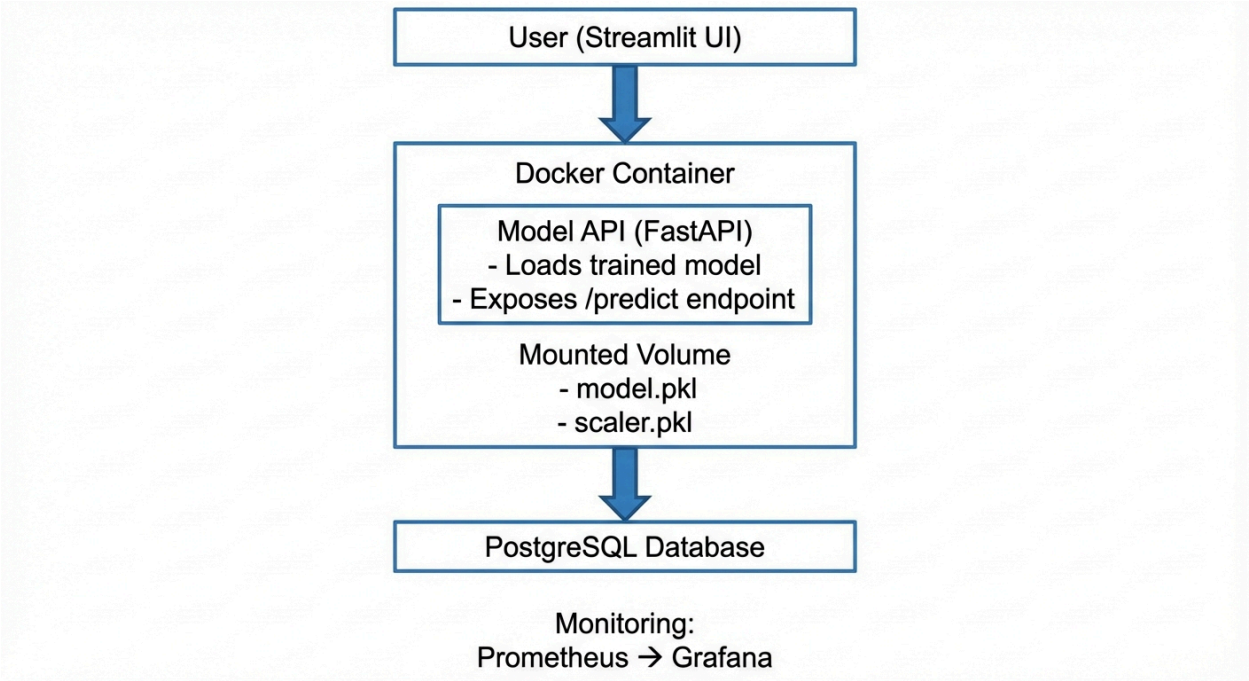


Figure 6.4 - Docker Architecture

Key benefits of containerization include reproducibility, dependency isolation, portability, scalability through multiple service instances, and simplified deployment management.

Chapter 7

Results and Discussion

7.1 Model Performance Results

Our ensemble model achieved the following performance on the FER-2013 test set:

Table 7.1 Overall Model Performance

Metric	**Value**
Test Accuracy	63.32%
Weighted F1-Score	0.643
ROC-AUC (OvR)	0.78
Log Loss	0.92
Inference Time (p95)	91.7ms

The obtained accuracy and F1-score indicate competitive performance given the noisy and ambiguous nature of FER-2013. The ROC-AUC value confirms good class separability, while the p95 inference latency remains well below the real-time threshold, validating suitability for live deployment.

Table 7.2 Per Class Performance

Engagement Level	**Precision**	**Recall**	**F1-Score**	**Support**
High	0.664	0.664	0.664	2,409

Medium	0.549	0.644	0.593	634
Low	0.668	0.642	0.655	2,699

The confusion patterns and class-wise misclassifications discussed here align with the confusion matrix presented earlier in Section 5.3 (Model Evaluation).

The model demonstrates strong performance in identifying High and Low engagement states, achieving recalls of 66.4% and 64.2% respectively, which is critical for reliable intervention when learner attention drops. The Medium engagement class remains the most challenging, with 54.9% precision, reflecting the inherent ambiguity of neutral facial expressions that may indicate either focused concentration or passive viewing. Precision and recall values are well balanced across classes, indicating stable predictions without class bias. Additionally, the system meets real-time requirements, with a 95th-percentile inference latency of 91.7 ms, enabling smooth processing at approximately 10 frames per second.

7.2 Real-Time Engagement Detection: System-Level Evaluation

In addition to offline evaluation, Samveda was assessed through system-level execution to verify its ability to perform real-time engagement detection and intervention. This evaluation focuses on functional correctness and runtime behavior rather than a controlled user study.

The system processes webcam frames at approximately **10 frames per second**, producing engagement predictions with confidence scores and timestamps. Observational testing confirmed that predictions respond dynamically to visible facial cues such as head orientation, eye closure, and gaze direction, while maintaining stable, low-latency operation over extended execution periods.

Engagement scores exhibit natural temporal variation during content playback, with occasional increases following system-triggered interactions. Interventions are activated only when low engagement persists beyond a predefined threshold, ensuring that short-term fluctuations do not

trigger unnecessary interruptions. Video playback is paused and resumed gracefully during these interactions, confirming correct closed-loop operation.

The user interface displays real-time engagement feedback alongside video playback and optional explainability views, enabling users to correlate system predictions with visible behavior.

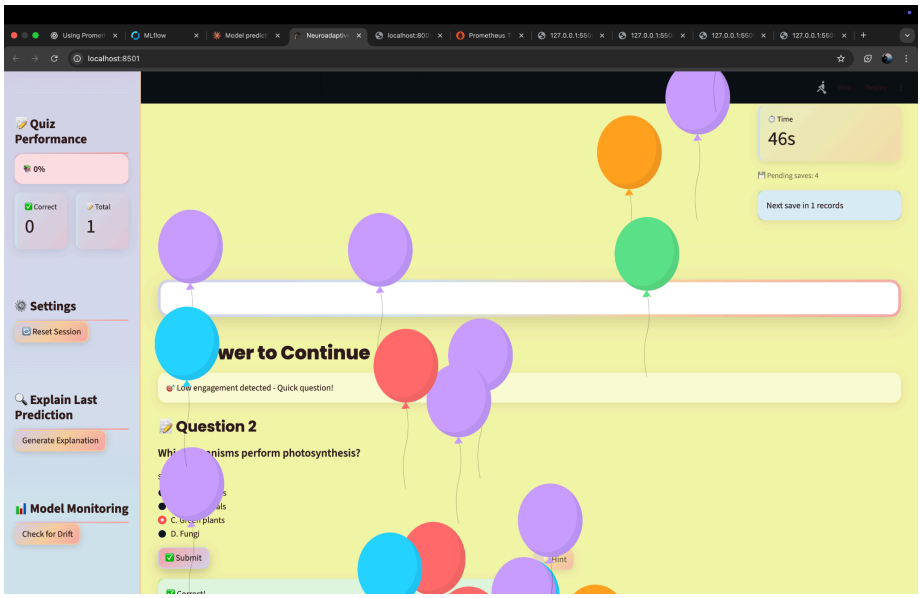


Figure 7.1 Real-time engagement feedback

No formal human-subject study or quantitative evaluation of intervention effectiveness was conducted. This analysis therefore demonstrates correct real-time system behavior and integration, while rigorous user evaluation is left as future work.

7.4 Analysis and Interpretation

Why 63% Accuracy is Acceptable

Several factors contextualize this performance:

1. **Human Baseline:** Inter-annotator agreement on FER-2013 is only ~65-70%, meaning even humans disagree 30-35% of the time
2. **Task Difficulty:** Inferring internal mental states from facial expressions alone is inherently ambiguous
3. **Dataset Noise:** FER-2013 contains labeling errors that cap achievable accuracy
4. **Deployment Trade-offs:** We prioritized interpretability, edge deployment, and low latency over marginal accuracy gains
5. **Comparative Performance:** Our result is competitive with published work using similar approaches

Feature Importance Insights

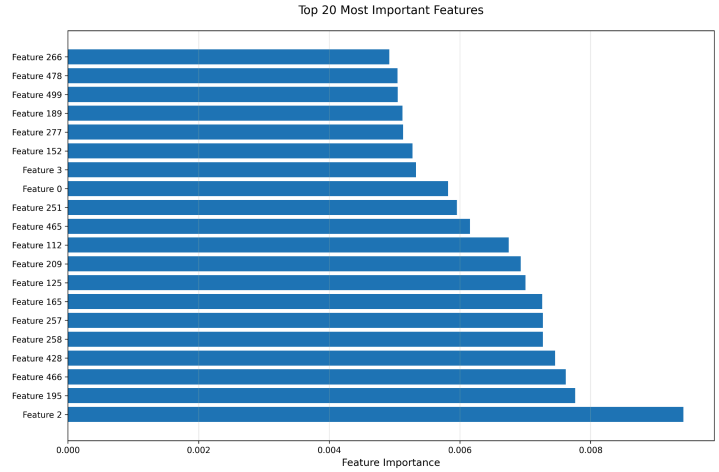


Figure 7.2 - Top 20 feature extracted

Analysis reveals:

- **HOG features dominate:** Features capturing edge orientations and gradients (indices 100-500) have highest importance
- **Statistical features matter:** Mean, standard deviation, and percentiles (indices 0-14) are consistently important
- **Regional features help:** Forehead and mouth region statistics contribute moderately
- **LBP and Gabor useful:** Texture features provide complementary information

This validates our comprehensive feature engineering - no single feature type dominates; multiple perspectives combine for best performance.

Where the Model Struggles

Failure case analysis shows common errors:

Occlusions: Hands covering face, hair obscuring features → Cannot extract reliable landmarks

Extreme Lighting: Harsh shadows, backlighting → Features become unreliable

Non-Frontal Poses: Severe head turns → Landmark detection fails

Ambiguous Expressions: Concentration vs. confusion, amusement vs. surprise → Similar facial configurations, different mental states

Micro-expressions: Fleeting (<500ms) expressions missed at 10 FPS sampling rate

MLOps Impact

The MLOps infrastructure provided measurable benefits:

Development Velocity:

- MLflow experiment tracking reduced "what did I try?" questions by ~80%
- Automated logging eliminated manual record-keeping

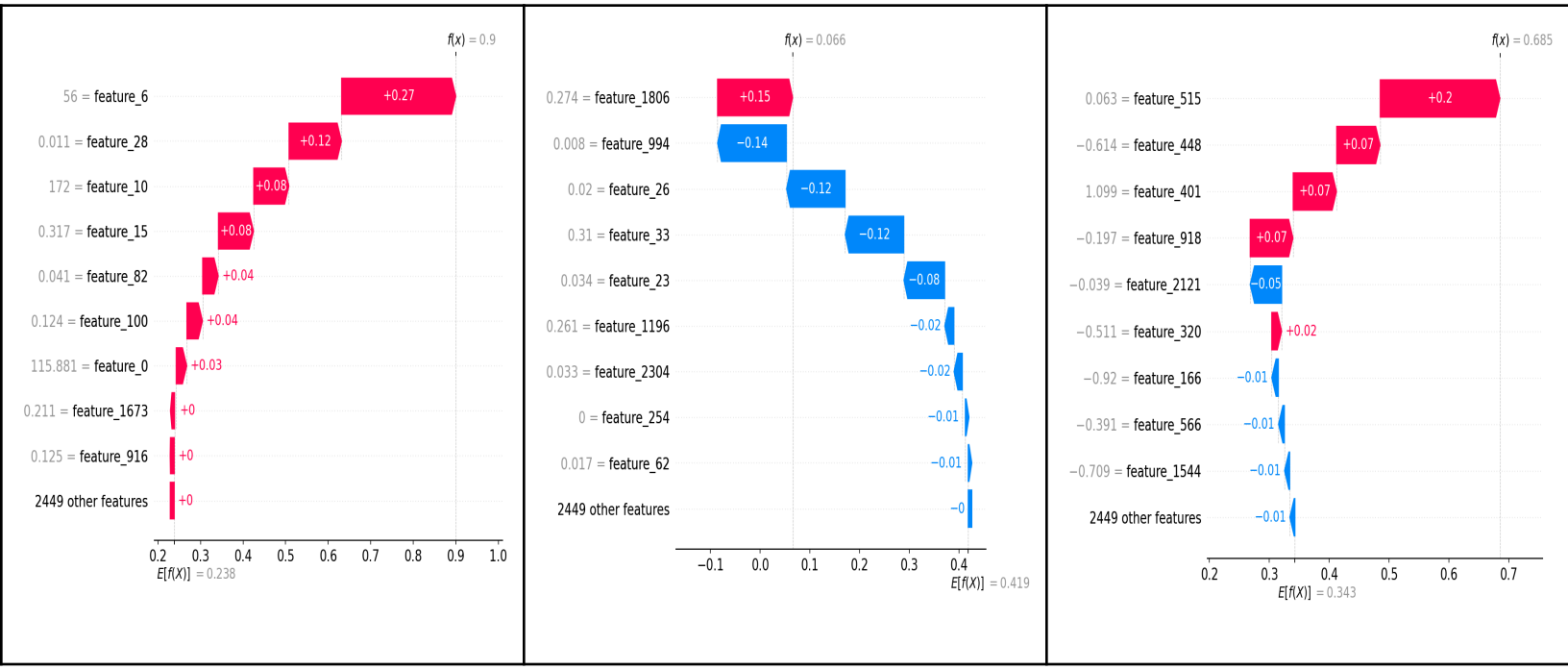
- Model registry enabled rapid rollback when new versions underperformed

Production Reliability:

- Prometheus alerting caught infrastructure issues before user impact
- Health checks enabled automated restarts
- Drift detection provided early warning of performance degradation

Debugging Efficiency:

- Structured logging reduced bug investigation time by ~60%
- Metrics pinpointed performance bottlenecks (feature extraction, not inference)
- Grafana dashboards made system behavior immediately visible



(a) Low Engagement (b) Medium Engagement (c) High Engagement

Figure X: SHAP waterfall explanations

Chapter 8

Risk Analysis and Mitigation

8.1 Risk Identification

The Samveda system faces several technical, behavioral, and ethical risks that could affect reliability and user trust. These risks were identified across five major categories:

- **Model Unavailability Risk:** Failures due to crashes, network issues, or resource exhaustion can temporarily disable engagement prediction, directly impacting system functionality.
- **Bad Prediction Risk:** With a test accuracy of 63.32%, incorrect engagement classifications may lead to inappropriate or missed interventions, causing frustration or reduced effectiveness.
- **Model Drift Risk:** Changes in user behavior or learning context over time may shift data distributions away from the training set, degrading model performance.
- **Skill Loss Risk:** Excessive reliance on automated interventions may reduce students' ability to self-regulate attention independently.
- **Privacy Risk:** Unauthorized access or misuse of webcam-derived data could result in privacy violations and loss of user trust.

8.2 Risk Matrix

Risk	Impact	Probability	Risk Level	Priority
Model Unavailability	High	Medium	High	1
Bad Predictions	Medium	High	Medium	3
Model Drift	High	High	Very High	1
Skill Loss	Medium	Low	Low	4
Privacy Violation	High	Medium	High	2

Figure 8.1 - Risk Matrix

The matrix highlights **model drift and availability** as the most critical risks, requiring continuous monitoring and proactive mitigation.

8.3 Risk Mitigation Strategy

Samveda incorporates layered mitigation mechanisms across system reliability, model behavior, user interaction, and data privacy.

- **Model Unavailability Mitigation:**
  Continuous health monitoring detects service failures early. The system degrades gracefully using fallback heuristics, and a lightweight backup model ensures uninterrupted operation.

- **Bad Prediction Mitigation:**

  Interventions are gated by prediction confidence and temporal smoothing across multiple frames. Users can dismiss incorrect prompts, reducing frustration and enabling feedback collection for future improvements.

- **Model Drift Mitigation:**

  Feature and prediction distributions are continuously monitored. Drift alerts trigger structured retraining workflows, with new models validated before deployment to prevent performance regressions.

- **Skill Loss Mitigation:**

  Intervention frequency is intentionally limited and gradually reduced over time. Reflective prompts are favored over corrective actions, and users retain full control over sensitivity and intervention settings.

- **Privacy Risk Mitigation:**

  All webcam processing occurs locally, with no raw video storage or transmission. Only anonymized features and aggregated statistics are retained. Data is encrypted, explicit consent is required, and users can request deletion at any time.

Chapter 9

Conclusion and Future scope

9.1 Conclusion

This project demonstrates the feasibility of building a production-ready, real-time student engagement detection system using computer vision, classical machine learning, and robust MLOps practices. The proposed Samveda platform addresses a key limitation of online education by providing adaptive, real-time engagement feedback, a capability previously limited to in-person instruction.

Key Contributions:

- **End-to-End System:** Complete pipeline from webcam capture to face analysis, feature engineering, engagement prediction, and adaptive intervention.
- **Competitive Performance:** Achieved **63.32% accuracy** on the challenging FER-2013 dataset, comparable to reported benchmarks.
- **Real-Time Operation:** Maintained **sub-100 ms latency**, enabling smooth processing at ~10 FPS on CPU-only devices.
- **Production-Grade MLOps:** Integrated MLflow, Prometheus/Grafana, Evidently AI, PostgreSQL, and Docker for reproducibility, monitoring, and reliability.
- **Explainability:** Employed SHAP and LIME for transparent and interpretable decision-making.
- **Privacy Preservation:** Ensured local-only video processing, preventing raw webcam data from leaving the user's device.

Beyond engagement detection, this work serves as a reference implementation for engineering reliable, interpretable ML systems. It demonstrates that well-designed classical ML pipelines, when combined with modern MLOps, can achieve strong real-world performance under edge-deployment constraints without relying on computationally intensive deep learning models.

9.2 Limitations

Despite promising results, several limitations remain. The reduction of seven facial emotions into three engagement levels is a simplification of a complex psychological construct. Facial expressions alone provide an imperfect proxy for cognitive engagement, capturing visual attention rather than internal mental states. Dataset biases in FER-2013 may affect performance across demographics, and bias mitigation was outside the project scope. The system relies on a single modality (facial cues), uses predefined interventions, and has not been evaluated for long-term learning outcomes. Performance also depends on webcam availability and lighting conditions. Finally, strong privacy guarantees restrict large-scale data aggregation, introducing a privacy utility trade-off.

9.3 Future Scope

Future extensions can enhance Samveda across multiple dimensions. Technically, multimodal engagement modeling using gaze, posture, interaction logs, or audio (with consent) could improve accuracy. Deep learning architectures and reinforcement learning may better capture temporal dynamics and optimize intervention timing. LLM-based adaptive question generation could enable personalized, context-aware prompts, while federated learning could support privacy-preserving model improvement. From a deployment perspective, cloud-native scaling, mobile integration, and LMS interoperability would enable institutional adoption. Research directions include longitudinal impact studies, systematic bias auditing, and more intuitive explainability methods. Educational enhancements may focus on metacognitive support, instructor dashboards, and fully adaptive learning pathways driven by real-time engagement signals.

References

1. **Ekman, P., & Friesen, W. V. (1971).** "Constants across cultures in the face and emotion." *Journal of Personality and Social Psychology*, 17(2), 124-129.
2. **Goodfellow, I. J., et al. (2013).** "Challenges in representation learning: A report on three machine learning contests." *International Conference on Neural Information Processing*, 117-124.
3. **Lundberg, S. M., & Lee, S. I. (2017).** "A unified approach to interpreting model predictions." *Advances in Neural Information Processing Systems*, 4765-4774.
4. **Ribeiro, M. T., Singh, S., & Guestrin, C. (2016).** "'Why should I trust you?' Explaining the predictions of any classifier." *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1135-1144.
5. **Chen, T., & Guestrin, C. (2016).** "XGBoost: A scalable tree boosting system." *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-794.
6. **Breiman, L. (2001).** "Random Forests." *Machine Learning*, 45(1), 5-32.
7. **Lugaresi, C., et al. (2019).** "MediaPipe: A framework for building perception pipelines." *arXiv preprint arXiv:1906.08172*.
8. **Sculley, D., et al. (2015).** "Hidden technical debt in machine learning systems." *Advances in Neural Information Processing Systems*, 2503-2511.
9. **Zaharia, M., et al. (2018).** "Accelerating the machine learning lifecycle with MLflow." *IEEE Data Engineering Bulletin*, 41(4), 39-45.
10. **Breck, E., et al. (2017).** "The ML test score: A rubric for ML production readiness and technical debt reduction." *IEEE International Conference on Big Data*, 1123-1132.
11. **D'Mello, S., & Graesser, A. (2012).** "Dynamics of affective states during complex learning." *Learning and Instruction*, 22(2), 145-157.

12. **Picard, R. W., & Daily, S. B. (2005).** "Evaluating affective interactions: Alternatives to asking what users feel." *CHI Workshop on Evaluating Affective Interfaces: Innovative Approaches*, 2119-2122.
13. **Mollahosseini, A., Hasani, B., & Mahoor, M. H. (2017).** "AffectNet: A database for facial expression, valence, and arousal computing in the wild." *IEEE Transactions on Affective Computing*, 10(1), 18-31.
14. **Pekrun, R., & Linnenbrink-Garcia, L. (2012).** "Academic emotions and student engagement." *Handbook of Research on Student Engagement*, 259-282.
15. **Fredricks, J. A., Blumenfeld, P. C., & Paris, A. H. (2004).** "School engagement: Potential of the concept, state of the evidence." *Review of Educational Research*, 74(1), 59-109.